

**Expected Start Date: 29th September, 2025**

**Expected End Date: 30th June, 2026**

**Meeting: Alternative weeks starting on Oct 3 - Friday's at 10 AM**

**Work I got:** Updating the Data Engineering curriculum

Data Engineering Course Winter 2025 - <https://sites.google.com/pdx.edu/dataengspring2025>

Trello Board: <https://trello.com/b/QakdD47W/dataeng-improvements>

Slack (Alex) - <https://app.slack.com/client/T0199EMRRMM/D08L1945Z61>

GitLab (Data Eng Repo): <https://gitlab.cecs.pdx.edu/nharris/dataeng-server>

Sept 22, 27 (week 1), I worked on creating repo's, organizing code, getting familiar with the tools.

### What did Prof. Bruce share with me?

1. Old System Architecture designs
2. Trello board
3. Ideas for Projects - doc

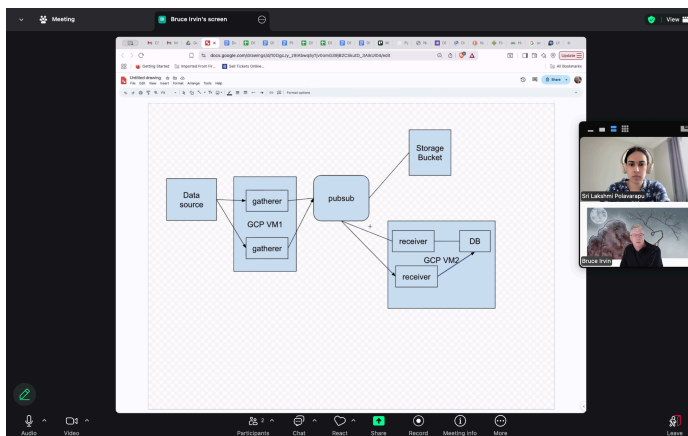
### Task 1: System Architecture

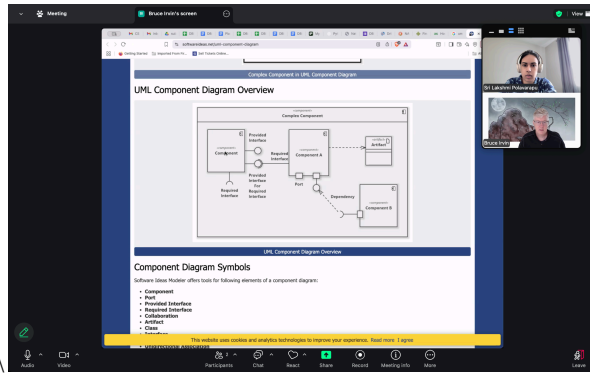
- Working on designing architectures for all the parts of the project, and there should be one full architecture.
- <https://app.diagrams.net/?src=about> (4 diagrams)

### Meet 1: Sept 29, Sept 30, Oct 3 (week 2)

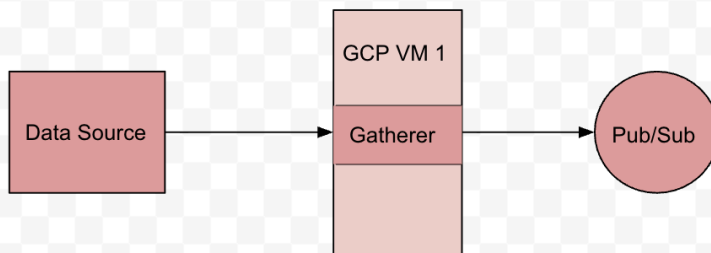
(continued doing this the whole week)

- Prof, wants component diagrams
- Explain what each component does.
- Use 3 different colors for 3 parts
- No need for the part 4 diagram
- Below is the screenshot of what Prof, explain - how the component diagram should be.

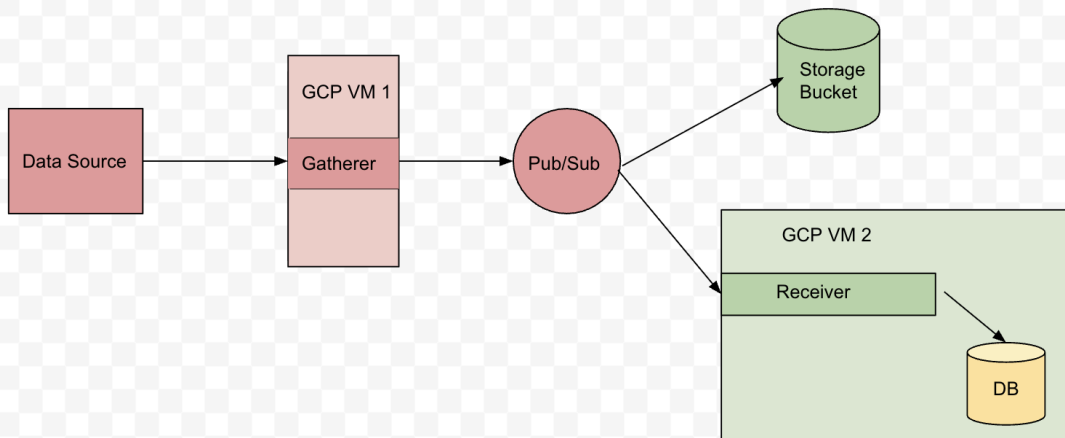




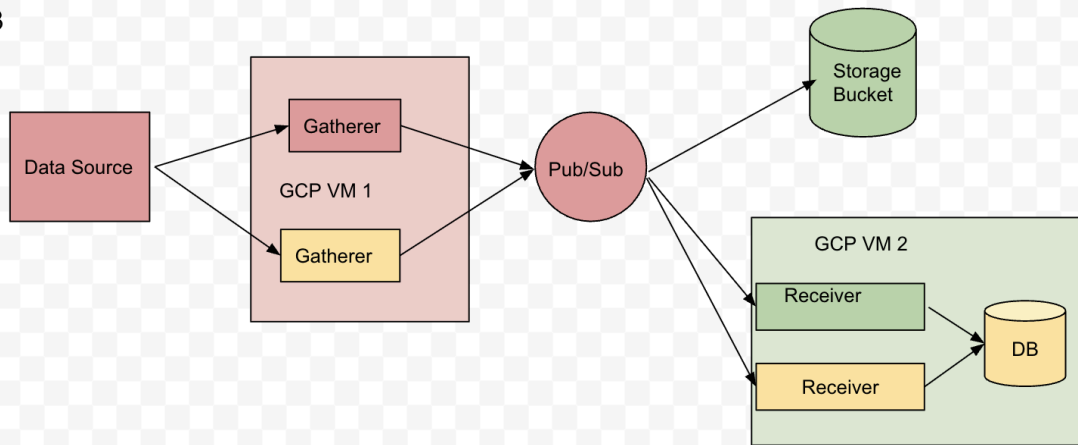
## Part 1



## Part 2



### Part 3



### Task 2: Run the existing server again

- created a new VM

### Oct 7th, Oct 9th (Alex) (week 3)

- Contacted Alex and gathered some information about the server
- I needed to contact the CAT team, and emailed Bruce about it.
- Alex added me to the Data Eng Git repo - MCECS login

Working on running the existing one:

1. Got the VPN working
2. Cloned the data eng ser repo (gave by Alex)
3. Installed the required libraries
4. What should I change the date to? Nano startdate.json
5. I am not able to run step6 commands - permission denied
6. Port 8000 already in use, need to contact Alex

### Few commands to keep in mind:

```
ssh srilakp@busdata.cs.pdx.edu
```

```
cd ~/.config/systemd/user/dataeng-server.service
```

### Oct 13th, Oct 17th (Alex meet) (week 4)

- Installing collected packages: pytz, tzdata, six, packaging, numpy, markupsafe, itsdangerous, click, blinker, werkzeug, python-dateutil, jinja2, gunicorn, pandas, flask
- server\$ systemctl --user start dataeng-server
- server\$ systemctl --user status dataeng-server
- curl -XGET "https://busdata.cs.pdx.edu/api/dataDateCheck"
- loginctl show-user \$(whoami) | grep Linger
- Nano startdate.json

### I need to be in the dataeng-server path and do the step 6 (from gitlab)

This is step 6—> Copy both the breadcrumb data and the html page data to the static and templates folder with the following commands from within the repo directory (i.e. `~/dataeng-server/`)

- `cp -r /stash/dataeng/trimet_data/s24/bc/* static/bc/.`
- `cp -r /stash/dataeng/trimet_data/s24/se/se_html/* templates/se_html/.`
- `cp -r /stash/dataeng/freeway_data/s25/freeway_data_split static/freeway_data_split`

Need to contact CAT again for requesting access to dataeng server - **Done**

- `srilakp@busdata:~/dataeng-server$ cp -r /stash/dataeng/trimet_data/s24/bc/* static/bc/.`
- `cp: cannot stat '/stash/dataeng/trimet_data/s24/bc/*': Permission denied`

### Oct 23rd, Oct 24th (week 5)

1. **Create 2 separate VMs and make them work**
2. **Change the diagram a little bit**
3. **Gather data properly from the local terminal**

**Steps to run in the terminal for gathering the data:**

- `ls ~/dataeng-server/static/bc/bc_2022-12-05`
- `ls ~/dataeng-server/templates/se_html`
- `ls ~/dataeng-server/static/freeway_data_split`
- `curl https://busdata.cs.pdx.edu/api/getBreadCrumbs?vehicle_id=2901`

### Oct 28, Oct 29 (week 6)

**Created a new directory - new\_dataeng**

#### Things I did in VM1:

- `python3 -m venv venv`
- `source venv/bin/activate`
- `pip install google-cloud-pubsub requests pandas`
- `gcloud auth application-default login`
- `Run data_gather.py`

### Nov 5, Nov 7 (week 7)

#### Things I did in VM2:

- `python3 -m venv venv`
- `source venv/bin/activate`
- `pip install google-cloud-pubsub psycpg2-binary pandas`
- `gcloud auth application-default login`
- `sudo nano /etc/systemd/system/subscriber.service`
- `sudo systemctl daemon-reload`
- `sudo systemctl enable subscriber.service`
- `sudo systemctl start subscriber.service`
- `sudo systemctl status subscriber.service`
- `sudo journalctl -u subscriber.service -f`



## 10th Nov, (week 8)

Alex took back the server access but told me to use the local method or linux method. Local method - I did not have enough space to download the file.

## 17th Nov, (week 9)

### To-Do List

1. Count the number of credits used per day and per week for 200 vehicles, 300 vehicles, 676 vehicles.
2. Observe how much longer the pipeline runs for 200 vehicles, 300 vehicles, 676 vehicles.
3. Check if the data is getting added to DB

## 200 Vehicles

**Task 4: Keep track of GCP credits** (check the credits after taking 300 vehicles)

## 18th Nov, (week 9)

### Checking if the pipeline is running fine?

1. Executed `data_gather.py`, which made the backlog spike (pub/sub health) go up.
2. Executed `subscriber.py`, which made the backlog spike go down.
3. This command `-> sudo systemctl status subscriber.service`, is actively running.

-> working on subscriber script, as it is not publishing messages as the reason, not pushing data into DB.

-> there's no DB in my VM, so I'm working on that.

### Solution:

-> Though the subscriber published some messages and DB has data in it, I need to check it tomorrow again, because when I ran again, it gave me a "no messages found" message.

Total data - 634741

296 - trip table in DB

17780 - breadcrumb table in DB

→ After work, credit = \$39.85

## 19th Nov, (week 9)

Latest script for subscriber is `subscriber5.py`

Credits at the beginning of the work = \$39.34

Total data - 668117

Rows in trip = 1171

Rows in breadcrumb = 21463

Time `data_gather` pipeline takes is 2.42, 2.45 minutes

Time Subscriber pipeline takes is 9 min

## 24nd Nov, (week 10)

Total data= 496965

This table is for 200 credits

| Date     | Begin   | End     | Compute Engine | Pub/sub     | VM Manager | Hours worked | Time taken        | DB           | Total data |
|----------|---------|---------|----------------|-------------|------------|--------------|-------------------|--------------|------------|
| 18th Nov | \$39.85 | \$39.34 | \$0.496        | 0.128 cents | \$0.036    | 4 hours      |                   |              |            |
| 19th Nov | \$39.34 | \$39.32 | \$0.32         | \$0.001     | \$0.024    | 3 hours      |                   |              |            |
| 20th Nov |         |         | \$0.09         | \$0.009     |            | 0            |                   |              |            |
| 21st Nov |         | 38.79   | \$0.09         | \$0.014     |            | 0            |                   |              |            |
| 22nd Nov | 38.79   |         |                |             |            |              | 1.92 min + 13 min | 1253 + 80269 | 496965     |
| 25th     |         |         |                |             |            |              | 2.5 min + 8 min   |              | 676194     |
| 27th     |         |         | \$0.195        | \$0.026     | \$0.018    | 1.5          | 2.25 + 12.12      |              | 677252     |

## 300 Vehicles

**Task 3: Take 300 vehicles** (--get to know more about this point)

- In the code, instead of using 200 vehicles (Coco), use 300 this time

**26th Nov, (week 10)**

1. Tried with 300 vehicles, pipeline running time increased.

[illegible]

### 30th Nov, (week 11)

Issue: data\_gather is taking longer to publish the records, and as per ChatGPT, VM is overload - Solved

### 5th Dec, (week 11)

- When running with 300 vehicles, I am getting a 'killed' error - **Solved**, updated the code.
  - **Solution:** VM1 ran out of memory when updated from 200 to 300 vehicles. I made sure, all the data is not loaded at once. And reads one file, publishes, deletes and moves on.
- Data was not inserted into DB - working on it - trip table is working fine, but breadcrumb table is not updating.
- Get new batch of data (trello)

### 8th Dec, (week 12)

Remaining Credits: \$36.31

300 vehicles,

Total breadcrumbs: 951854

Data gather took: 3.5 min

Subscriber took:

Scripts: dec2\_gather.py & dec2\_subscriber.py

676 vehicles

Total breadcrumbs: 2207723

Data gather took: 6.65 min

Subscriber took:

Scripts: dec2\_gather.py & dec2\_subscriber.py

### 12th Dec, (week 12)

Full day worked on making the subscriber work for large amounts of data (300, 676) - not done

→ used \$0.86 compute engine, \$0.086 pub/sub

### 13th Dec, (week 12)

**To-Do:** Got clarification from Alex, subscribed messages count must be equal to the published messages count.

Error encountered: While I was working on making the subscriber work, I suddenly noticed time out errors from the bus data API, and emailed the professor about it.

→ used \$0.98 credits (3rd Dec + 4th Dec)

System crash, confirmed from Alex. need to wait until Alex restarts the server.

### 15th Dec, (week 13)

Alex fixed the server.

Again working on making the subscriber receive an exact amount of data.

Running 676 vehicles currently.

Need to remove certain things before re-running

- `rm -rf bus_data`
- `rm -rf processed_data/*`
- `rm -rf extracted_json/*`
- `find . -type d -name "__pycache__" -exec rm -rf {} +`

Also, delete and create subscriber in cloud terminal, not in VM terminal

- gcloud pubsub subscriptions delete MyTopic1-sub
- gcloud pubsub subscriptions create MyTopic1-sub --topic=MyTopic1

Run subscriber first and then gather\_data script.

As of Dec 5th, scripts are —> **dec5\_gather.py** and **dec5\_sub.py**

For 676 before final code:

|         |         |           |           |
|---------|---------|-----------|-----------|
| Dec 5th | 1575184 | 24 min    | 35 min    |
| Dec 6th | 1572216 | 23.43 min | 37.08 min |

## 200 Vehicles

|                |               |                |                  |
|----------------|---------------|----------------|------------------|
| <b>Dec 7th</b> | <b>636536</b> | <b>8.6 min</b> | <b>17.19 min</b> |
|----------------|---------------|----------------|------------------|

## 300 Vehicles

|                |               |                  |                  |
|----------------|---------------|------------------|------------------|
| <b>Dec 7th</b> | <b>881358</b> | <b>12.77 min</b> | <b>22.15 min</b> |
|----------------|---------------|------------------|------------------|

## 676 Vehicles

|                |                |                  |               |
|----------------|----------------|------------------|---------------|
| <b>Dec 7th</b> | <b>2213617</b> | <b>30.49 min</b> | <b>55 min</b> |
|----------------|----------------|------------------|---------------|

### Dec 16th, (week 13)

As of today, both the scripts are working.

scripts are —> **dec5\_gather.py** and **dec6\_sub.py** (updated in the github)

### Dec 21st, (week 14)

The professor told the server the crash happened. He said it might be because of me accessing too many vehicles and also I might have opened too many connections and not closed them.

#### To-Do:

1. Check for any opened connections, if so close them.
2. Redo part-1, and suggest any improvements (prof, suggested high-level diagram for part1)

### Dec 22nd, (week 14)

## **Improvements/ suggestions in Part 1: (I thought of)**

### **1. Order of execution**

### **2. code responsibilities**

- Publisher - fetch + publish
- Subscriber - receive + save

### **3. gather script**

- Fetches 200 vehicles
- publishes everything in one go
- Holds resources for a long time

Prob: If at 150 th vehicle there's a problem then the whole run is a waste, need to re-run again

### Solution:

Sol 1: have multiple cron jobs but more complexity and not Efficient.

Sol 2: When gathering starts, it checks which vehicle ID's have already been processed - if Yes, mark it as done and when there's a failure, we can just run a few.

4. Did the gatherer successfully collect all the expected data for today? - Add a small Code - Like expected & processed

## **Jan 2, Jan 3, (week 15)**

### **Part1: I got another idea to be added in the doc (I faced this one).**

1. Do I need to separate gather and publisher into separate actions? But later I found out that gather must pass the data to the publisher to then publish each breadcrumb record.
2. Gather and publisher combined into a single program.
3. Using a logging module will be good. (i used it even before telling me)

## **Jan 6 (week 16)**

### **Improvements/ suggestions in Part 2: (I thought of)**

1. It might be good if we have validation checks before pushing data into the database. So usually we must have good quality data in the database, so I thought it would be good if we check for any bad timestamps, missing fields, weird GPS values before inserting data into PostgreSQL tables.
2. Should my database row count must match the input count, is the question I always had.
3. I put all assertions in one function, but it is suggested to put each assertion in one function.
4. Better to have subscriber in a infinite loop, so that it won't start & stop constantly via systemd

## **Jan 9, (week 16)**

### **Improvements/ suggestions in Part 3: (I thought of)**

1. *Before finalizing a visualization, verify that the selected trip truly meets the geographic and route constraints by checking route\_id, trip\_id, and bounding box intersections directly in SQL.*
2. *In the document it was mentioned to use classes, functions, and basic OOPs. so we can ask students what exactly they used, why did they use this?*
3. In the doc, we mentioned building a new pipeline that operates just like the previous one, so we can add a sentence like - The Stop Events pipeline may reuse the same architectural pattern and helper utilities as the

Breadcrumb pipeline, but should be implemented as a separate logical pipeline using modular classes and functions.

### **Jan 10th, (week 16)**

I worked on setting up the server again, but there was a crash in the server by another student, so it took a while to correct that. I re-run everything from the start, but using my already existing code scripts. I did this because the Professor told me to give an update on what problems are being faced while running the whole pipeline again, so it can be helpful for winter 2026 students.

**Task 5: After Prof. gives me new updated Trimet data, I need to work on setting up the server again.**

### **Jan 13th, (week 17)**

My Professor and I had a meeting, so we discussed what improvements I thought and shared it with him. He liked some of the points I told.

He told me a new set of trimet data, but they have some inconsistencies so he gave me work to identify those mistakes, so he can let the person who gave the data know about the issues so he can give us refined data.

### **To-Do**

1. Set cron job - not recommended now, will think about this later.

### **23rd Jan, 24th Jan (week 18),**

I had a long meeting with the professor, explaining every improvement / suggestion I thought of. Also he told me to check the new TriMet data given by Saba. And he told me what to check exactly.

1. Missing fields
2. Identify the data
3. Check, how to identify the issues in the data
4. Check breadcrumb records
5. Check StopEvent records
6. Check columns

Location he told me to check is this: `/stash/dataeng/trimet_data/Summer2025`

The professor told me to check it using Babbage too.

### **Jan 26th, Jan 27th(week 19)**

I started working on new data but there is a permission access issue I'm facing.

```

[srilakp@busdata:/$
[srilakp@busdata:/$
[srilakp@busdata:/$ cd /stash/dataeng/trimet_data
[srilakp@busdata:/stash/dataeng/trimet_data$ ls
0_og_data s23_archive s24 Summer2025
[srilakp@busdata:/stash/dataeng/trimet_data$ cd Summer2025
-bash: cd: Summer2025: Permission denied
[srilakp@busdata:/stash/dataeng/trimet_data$
[srilakp@busdata:/stash/dataeng/trimet_data$
[srilakp@busdata:/stash/dataeng/trimet_data$
[srilakp@busdata:/stash/dataeng/trimet_data$ cd /stash/dataeng/trimet_data/Summer2025
-bash: cd: /stash/dataeng/trimet_data/Summer2025: Permission denied
[srilakp@busdata:/stash/dataeng/trimet_data$ █

```

Using Babbage also I tried.

```

[srilakp@babbage:~$ cd /stash/dataeng/trimet_data/Summer2025
-bash: cd: /stash/dataeng/trimet_data/Summer2025: Permission denied
[srilakp@babbage:~$ cd /stash/dataeng/trimet_data
[srilakp@babbage:/stash/dataeng/trimet_data$ ls
0_og_data s23_archive s24 Summer2025
[srilakp@babbage:/stash/dataeng/trimet_data$ cd Summer2025
-bash: cd: Summer2025: Permission denied
[srilakp@babbage:/stash/dataeng/trimet_data$ █

```

Contacted the professor about the issue, and also need to contact the CAT team.

The professor got back to me with permission, now it's working.

```

* Phone:      503-725-5420
* Chat:       https://support.cat.pdx.edu
* Location:    FAB 82-01

[srilakp@babbage:~$ cd /stash/dataeng/trimet_data/Summer2025
[srilakp@babbage:/stash/dataeng/trimet_data/Summer2025$ ls
Summer25BC.csv Summer25CAD_AVL.csv
[srilakp@babbage:/stash/dataeng/trimet_data/Summer2025$ █

```

**Feb 3rd, Feb 6th, Feb 7th (week 20)**

```

[srilakp@babbage:/stash/dataeng/trimet_data/Summer2025$ ls
Summer25BC.csv Summer25CAD_AVL.csv
[srilakp@babbage:/stash/dataeng/trimet_data/Summer2025$ █

```

Both csv's are there.

## STEP 1

**Summer25BC**

Size: 75 MB

Rows: 1,112,305

7 headers:

VEHICLE\_NUMBER,  
ARRIVE\_DATETIME,  
SERVICE\_DATE,  
LATITUDE,  
LONGITUDE,  
SPEED,  
DWELL

Spot checks and partial scans did not reveal any inconsistencies in the number of columns per row

### **SummerCAD\_AVL**

Size: 3.0 GB

Rows: 24,573,932

26 headers:

SERVICE\_DATE,VEHICLE\_NUMBER,LEAVE\_TIME,TRAIN,BADGE,ROUTE\_NUMBER,DIRECTION,SERVICE\_KEY,TRIP\_NUMBER,STOP\_TIME,ARRIVE\_TIME,DWELL,LOCATION\_ID,DOOR,LIFT,ONS,OFFS,ESTIMATED\_LOAD,MAXIMUM\_SPEED,TRAIN\_MILEAGE,PATTERN\_DISTANCE,LOCATION\_DISTANCE,X\_COORDINATE,Y\_COORDINATE,DATA\_SOURCE,SCHEDULE\_STATUS

Spot checks and partial scans did not reveal any inconsistencies in the number of columns per row

Can conclude from above: Files are readable, headers are correct, rows are structurally uniform, no obvious corruption

### **STEP 2: checking missing values & empty fields**



```

srilakp@babbage:/stash/dataeng/trimet_data/Summer2025$ awk -F',' '
NR>1 {
    for (i=1;i<=NF;i++)
        if ($i=="") missing[i]++
}
END {
    for (i=1;i<=NF;i++)
        printf "Column %d missing: %d\n", i, missing[i]+0
}' Summer25BC.csv
Column 1 missing: 0
Column 2 missing: 4
Column 3 missing: 0
Column 4 missing: 0
Column 5 missing: 0
Column 6 missing: 0
Column 7 missing: 0

```

The Bus Census dataset contains four records with missing arrival timestamps, while all other fields are complete.

```

srilakp@babbage:/stash/dataeng/trimet_data/Summer2025$ awk -F',' '
$NF==" " {count++}
END {print "Rows with empty last field:", count+0}
' Summer25BC.csv
Rows with empty last field: 0

```

```

srilakp@babbage:/stash/dataeng/trimet_data/Summer2025$ head -n 500000 Summer25CAD_AVL.csv | awk -F',' '
NR>1 {
    for (i=1;i<=NF;i++)
        if ($i=="") {bad++; break}
}
END {
    print "Rows with ≥1 missing field:", bad+0
}'
Rows with ≥1 missing field: 2270
srilakp@babbage:/stash/dataeng/trimet_data/Summer2025$ head -n 500000 Summer25BC.csv | awk -F',' '
NR>1 {
    for (i=1;i<=NF;i++)
        if ($i=="") {bad++; break}
}
END {
    print "Rows with ≥1 missing field:", bad+0
}'
Rows with ≥1 missing field: 0
srilakp@babbage:/stash/dataeng/trimet_data/Summer2025$ █

```

```

srilakp@babbage:/stash/dataeng/trimet_data/Summer2025$ awk -F',' '
$NF==" " {count++}
END {print "Rows with empty last field:", count+0}
' Summer25CAD_AVL.csv
Rows with empty last field: 0

```

**After step2, to conclude:** I checked whether any values are missing in the two CSV files. The Bus Census file (Summer25BC.csv) is almost complete, with only 4 rows missing arrival time out of more than 1 million rows, which means missing data is very rare and not a serious issue in this dataset. In contrast, the CAD/AVL file (Summer25CAD\_AVL.csv) shows a noticeable amount of missing data. In a sample of 500,000 rows, 2,270 rows had at least one missing value, which means missing data occurs regularly in this file. This indicates that the CAD/AVL dataset is not fully complete and will need data cleaning or handling of missing values for CAD dataset before it can be used reliably for further analysis.

**Feb 12, Feb 13 (week 21)**

## 1. Timestamp and date issues

```
srilakp@babbage:/stash/dataeng/trimet_data/Summer2025$ awk -F',' 'NR>1 {
  ad=substr($2,1,2)
  sd=substr($3,1,2)
  if (ad != sd && ad != sd+1)
    print NR, $2, $3
}' Summer25BC.csv | head
1322 07JUN2025:00:00:43 06JUN2025:00:00:00
1323 07JUN2025:00:01:14 06JUN2025:00:00:00
1324 07JUN2025:00:02:28 06JUN2025:00:00:00
1325 09JUN2025:00:03:11 08JUN2025:00:00:00
1326 07JUN2025:00:04:29 06JUN2025:00:00:00
1327 07JUN2025:00:05:02 06JUN2025:00:00:00
1328 09JUN2025:00:06:02 08JUN2025:00:00:00
1329 07JUN2025:00:07:04 06JUN2025:00:00:00
1330 07JUN2025:00:08:21 06JUN2025:00:00:00
1331 07JUN2025:00:09:06 06JUN2025:00:00:00
```

1. I compared ARRIVE\_DATETIME with the service date (SERVICE\_DATE) in the Bus Census data to see if they follow a consistent rule.
2. Some records have arrival times that occur after midnight on the next day, which is expected for late-night service.
3. Similar arrival times are sometimes linked to different service dates without a clear or consistent pattern. Like times just after midnight are assigned to the previous days.
4. So we get confused about which day which record belongs to.
5. Arrival timestamps look valid but their mapping to service dates are inconsistent.

## 2. Wrong time order

```

srilakp@babbage:/stash/dataeng/trimet_data/Summer2025$ awk -F',' '
NR>2 && $1==prev && $2 < prev_time {
    print NR, $1, $2, prev_time
}
{
    prev=$1
    prev_time=$2
}' Summer25BC.csv | head
3 00103 10JUL2025:00:01:17 11JUL2025:00:00:15
14 00103 10JUL2025:00:12:06 11JUL2025:00:11:58
37 00103 10JUL2025:00:35:29 23JUL2025:00:34:41
50 00103 10JUL2025:00:48:05 22JUL2025:00:47:13
54 00103 10JUL2025:00:52:36 22JUL2025:00:51:41
64 00103 23JUL2025:01:02:07 24JUL2025:01:01:05
66 00103 10JUL2025:01:04:06 23JUL2025:01:03:24
75 00103 10JUL2025:01:13:22 22JUL2025:01:12:43
81 00103 22JUL2025:01:19:50 23JUL2025:01:18:01
100 00103 23JUL2025:01:38:27 24JUL2025:01:37:26

```

I checked whether the arrival times for each vehicle appear in the correct time order in the file.

1. For the same vehicle, some records appear in the wrong order.
2. A record with an earlier arrival time sometimes appears after a record with a later arrival time. For example, an entry from July 10 appears after entries from July 11 or July 23 for the same vehicle.
3. When working with transit data, it is usually assumed that events for a vehicle are listed in time order. If the records are out of order, calculations such as speed, trip duration, distance traveled can give us incorrect answers.
4. Data needs to be sorted before doing any time-based analysis

It must be like this:

```

10JUL 00:00
10JUL 00:01
10JUL 00:02
10JUL 00:03

```

But,

Row 3: 10JUL2025:00:01:17

Row 2: 11JUL2025:00:00:15 (later time appears first)

### Feb 17th, Feb 18, Feb 19th, (week 22)

What does Summer25BC.csv have? Breadcrumb data, so each time the bus reports its location we get a new row, that happens every second.

### 3. GPS location jump check.



```

srilakp@babbage:/stash/dataeng/trimet_data/Summer2025$ awk -F',' '
NR>2 && $1==prev {
    dlat = $4 - prev_lat
    dlon = $5 - prev_lon
    dist = sqrt(dlat*dlat + dlon*dlon)

    if (dist > 0.05)
        print NR, $1, $2, $4, $5
}
{
    prev = $1
    prev_lat = $4
    prev_lon = $5
}' Summer25BC.csv | head
3 00103 10JUL2025:00:01:17 45.53574 -122.585232
10 00103 11JUL2025:00:08:02 45.529691 -122.931996
14 00103 10JUL2025:00:12:06 45.530111 -122.662539
36 00103 23JUL2025:00:34:41 45.580471 -122.57188
37 00103 10JUL2025:00:35:29 45.510707 -122.717244
49 00103 22JUL2025:00:47:13 45.516768 -122.868628
50 00103 10JUL2025:00:48:05 45.492911 -122.815601
63 00103 24JUL2025:01:01:05 45.528726 -122.602726
64 00103 23JUL2025:01:02:07 45.530119 -122.666073
66 00103 10JUL2025:01:04:06 45.512716 -122.857095

```

So based on the output, from one location to another in a second, the jump is not realistic. To conclude: **breadcrumbs are not ordered by time.**

#### 4. We are checking whether the dates are going backward?

```

srilakp@babbage:/stash/dataeng/trimet_data/Summer2025$ awk -F',' '
NR>2 && $1==prev && $2 < prev_time {
    print NR, $1, $2, prev_time
}
{
    prev = $1
    prev_time = $2
}' Summer25BC.csv | head
3 00103 10JUL2025:00:01:17 11JUL2025:00:00:15
14 00103 10JUL2025:00:12:06 11JUL2025:00:11:58
37 00103 10JUL2025:00:35:29 23JUL2025:00:34:41
50 00103 10JUL2025:00:48:05 22JUL2025:00:47:13
54 00103 10JUL2025:00:52:36 22JUL2025:00:51:41
64 00103 23JUL2025:01:02:07 24JUL2025:01:01:05
66 00103 10JUL2025:01:04:06 23JUL2025:01:03:24
75 00103 10JUL2025:01:13:22 22JUL2025:01:12:43
81 00103 22JUL2025:01:19:50 23JUL2025:01:18:01
100 00103 23JUL2025:01:38:27 24JUL2025:01:37:26

```

And yes, the dates are going backward. It started on 10JULY, but 23JULY came in between.

We generally expect the records to be listed in the order of how they happened. Again, to conclude: **breadcrumbs are not ordered by time.**

#### 5. Check the time between bus stops

```

srilakp@babbage:/stash/dataeng/trimet_data/Summer2025$ head -n 50000 Summer25CAD_AVL.csv | awk -F',' '
NR>1 && $12 > 600 {
    print NR, $2, $12
}
' | head
21 00302 1056
22 00302 1056
70 00302 1451
119 00302 1420
167 00302 1274
216 00302 1117
268 00302 1031
317 00302 1444
385 00303 911
386 00303 911

```

There's a huge difference in seconds between stops, this can be a data recording error.  
The bus appears to stay at the stop for 15-20 min, and this happened more times.  
Used Summer25CAD\_AVL dataset

## 6. Check the no activity of passengers .

```
srilakp@babbage:/stash/dataeng/trimet_data/Summer2025$ head -n 500000 Summer25CAD_AVL.csv | awk -F',' '
NR>1 && $16==0 && $17==0 {
    count++
}
END {
    print "Stops with no passenger activity:", count
}
'
```

Stops with no passenger activity: 320103

Used Summer25CAD\_AVL dataset  
Out of 500,000 records, 320,103 have no passenger activity.

**We might conclude that: some passenger activity is missing, or passenger count is not recorded, some stop events are logged even when there is no stop**

### Important issues that are needed to be reported:

1. Time between the stops is more (5)
2. No passenger captivity at most stops (6)
3. I see some inconsistent patterns. Like times just after midnight, times are assigned to the previous days (1)

## Feb 24th, (week 23)

*I worked on data analysis and compiled all findings from the checks done so far (Steps 1-6) into a structured summary document to share with the professor.*

### Issues found:

1. Summer25BC.csv - 4 missing ARRIVE\_DATETIME values out of 1,112,305 rows (very minor)
2. Summer25CAD\_AVL.csv - 2,270 rows with at least one missing value in a sample of 500,000 rows (needs cleaning)
3. Timestamps are inconsistent - records just after midnight are assigned to the previous service date
4. Breadcrumb records are not sorted by time - dates go backward for the same vehicle
5. GPS location jumps between records are unrealistic given the 1-second reporting interval
6. Unusually high dwell times at stops (15-20 min) in the CAD/AVL data - possible recording error
7. 64% of stops in CAD/AVL sample show no passenger activity

*Sent a summary email to the professor highlighting the three most critical issues: the time ordering problem, the dwell time anomaly, and the missing passenger data. The professor acknowledged and said the data inconsistencies are expected to some extent and to document them clearly for Saba.*

## Feb 26th (week 23),

*Over these two days I focused on doing a deeper analysis of the CAD/AVL dataset and began writing up a formal data quality report.*

#### **Feb 17 - CAD/AVL deeper checks:**

1. *Verified column data types - checked that numeric fields like SPEED, ONS, OFFS, ESTIMATED\_LOAD contain only numeric values and no unexpected string entries*
2. *Checked ROUTE\_NUMBER coverage - verified which routes appear in the Summer 2025 data vs what was expected, to make sure no routes were dropped*
3. *Checked DIRECTION values - confirmed they only contain expected values (0 or 1), no nulls or unexpected strings*
4. *Checked SERVICE\_KEY distribution - verified the spread of service key types (W = weekday, S = Saturday, U = Sunday/holiday) to confirm the data covers the expected service days*

*Commands used:*

→ `awk -F',' 'NR>1 {print $8}' Summer25CAD_AVL.csv | sort | uniq -c | sort -rn | head`

→ `awk -F',' 'NR>1 {print $7}' Summer25CAD_AVL.csv | sort | uniq -c`

#### **Feb 18 - Writing up the data quality report:**

*Started writing a structured report documenting all the data quality issues found, in a format that can be shared with Saba (the person who provided the data) so corrections can be made for future data drops.*

#### **Report sections:**

- *Section 1: File overview (size, row count, column count for both CSVs)*
- *Section 2: Missing value analysis (per column counts)*
- *Section 3: Timestamp and date inconsistency findings*
- *Section 4: Time ordering issues in breadcrumb data*
- *Section 5: GPS location anomalies*
- *Section 6: Dwell time and passenger activity anomalies in CAD/AVL*
- *Section 7: Recommended fixes and data cleaning steps*

*Shared draft of the report with the professor for review.*

### **March 3rd, March 5th (week 24)**

*Had a follow-up with the professor to go over the data quality report. Professor reviewed the findings and gave feedback on the report.*

*Professor's feedback:*

1. *The documentation is thorough and clear - he appreciated the structured format*
2. *He will forward the report to Saba so the data issues can be corrected for the next data drop*
3. *He suggested adding one more check - verifying that VEHICLE\_NUMBER values in Summer25BC.csv match the vehicle numbers seen in Summer25CAD\_AVL.csv to confirm both files cover the same fleet*

*Additional check done (vehicle number cross-referencing):*

*Extracted unique vehicle numbers from both files and compared them:*

→ `awk -F',' 'NR>1 {print $1}' Summer25BC.csv | sort -u > bc_vehicles.txt`

→ `awk -F',' 'NR>1 {print $2}' Summer25CAD_AVL.csv | sort -u > cad_vehicles.txt`

→ `comm -23 bc_vehicles.txt cad_vehicles.txt | wc -l # vehicles in BC but not in CAD`

→ `comm -13 bc_vehicles.txt cad_vehicles.txt | wc -l # vehicles in CAD but not in BC`

*Finding: The majority of vehicle numbers appear in both files, but there are a small number of vehicles present only in one dataset. This could indicate vehicles that only had breadcrumb data collected or were only active on certain service types. Flagged this as an additional minor inconsistency in the report.*

*Next Steps:*

1. *Wait for professor to forward findings to Saba and receive a revised data drop*
2. *Once cleaner data is available, begin the actual pipeline processing with the Summer 2025 data*
3. *Continue monitoring the existing pipeline on the server for any new crashes or issues*

## March 20th

I had a meeting with the Professor, showed all my work. He did tell some points to work on

1. I need to check if the issues i faced for the new data are also present in the old data (22-23)
2. Check if any important fields are missing and compare with old data
3. Check if any new fields are added
4. Check if GPS is working fine, visualize plot trips for corresponding routes. Middle, end data - GPS
5. Visualize frequency distribution of dwell types (15 - 20 min gap) for 5th point in the email i sent
6. Check if all columns are there
7. Double check the timings AM or PM?

## April 1st, 2026

**Task 1:** I need to check if the issues i faced for the new data are also present in the old data (22-23)

1. The GPS recording improved in the new data, but timestamp completeness was already good in old data too.
2. GPS coordinates are now fully complete, zero missing lat/lon in Summer2025
3. GPS quality fields (satellites, HDOP) no longer recorded at all, so no missing issue there either
4. 4 missing timestamps, which was not a problem in the old data at all.
5. New data has missing **ARRIVE\_DATETIME** - this didn't exist as a problem in old data at all (old data used **ACT\_TIME** which was always present)

Code:

(old data)

```
awk -F',' 'NR>1 {
    for (i=1; i<=NF; i++)
        if ($i=="") missing[i]++
    }
END {
    for (i=1; i<=NF; i++)
        printf "Column %d missing: %d\n", i, missing[i]+0
    }' /stash/dataeng/trimet_data/s23_archive/top75/the75v.csv
```

(new data)

```
awk -F',' 'NR>1 {
    for (i=1; i<=NF; i++)
        if ($i=="") missing[i]++
    }
END {
```



```

for (i=1; i<=NF; i++)
    printf "Column %d missing: %d\n", i, missing[i]+0
}' /stash/dataeng/trimet_data/Summer2025/Summer25BC.csv

```

## Task 2:

### Code:

```

awk -F',' 'NR>2 && $4==prev_vehicle && $6 < prev_time {
    print NR, $4, $6, prev_time
}
{
    prev_vehicle=$4
    prev_time=$6
}' /stash/dataeng/trimet_data/s23_archive/top75/the75v.csv | head -20

```

Time ordering issue **exists in both old and new data**

This is not a new problem introduced by Saba's data — it was already present in 22-23

```

srilakp@busdata:/stash/dataeng/trimet_data/Summer2025$ awk -F',' 'NR>2 && $4==prev_vehicle && $6 < prev_time {
    print NR, $4, $6, prev_time
}
{
    prev_vehicle=$4
    prev_time=$6
}' /stash/dataeng/trimet_data/s23_archive/top75/the75v.csv | head -20
91 4036 20617 57965
351 4036 21052 58822
707 4036 22227 60634
1063 4036 22821 61550
1419 4036 23538 62335
1775 4036 24368 68606
2131 4036 25013 63836
2487 4036 25637 65951
2843 4036 26348 69231
3199 4036 27915 64531
3555 4036 28787 67220
3911 4036 29511 63207
4267 4036 30096 65280
4623 4036 30994 67992
4801 4036 31808 56155
4979 4036 32505 57018
5337 3212 18437 19740
5469 3212 20532 21526
5515 3212 20253 20911
5558 3212 20023 20527

```

## Task 3:

### Code:

```

awk -F',' 'NR>2 && $4==prev_vehicle {
    dlat = $8 - prev_lat
    dlon = $7 - prev_lon
    dist = sqrt(dlat*dlat + dlon*dlon)
    if (dist > 0.05)
        print NR, $4, $6, $8, $7
}

```



```
{
prev_vehicle=$4
prev_lat=$8
prev_lon=$7
}' /stash/dataeng/trimet_data/s23_archive/top75/the75v.csv | head -20
```

Explanation:

**What I checked:** For the same vehicle, I compared the GPS coordinates of each record with the previous record. If the distance jumped more than 0.05 degrees (roughly 4-5 km) in one step, I flagged it as an unrealistic jump.

#### What I found in Old BC (22-23):

- GPS location jumps are present for multiple vehicles
- For example, vehicle 4036 jumps from longitude -122.495 to -122.476 to -122.408 within consecutive records — that is several kilometers in one second, which is physically not possible for a bus
- Some rows also show completely empty GPS coordinates (rows 173, 496, 602) which ties back to the 7,059 missing GPS values found in Check 1

#### What I found in New BC (Summer2025):

- Same unrealistic GPS jumps exist, as identified during week 22 analysis

```
srilakp@busdata:/stash/dataeng/trimet_data/Summer2025$ awk -F',' 'NR>2 && $4==prev_vehicle {
dlat = $8 - prev_lat
dlon = $7 - prev_lon
dist = sqrt(dlat*dlat + dlon*dlon)
if (dist > 0.05)
print NR, $4, $6, $8, $7
}
{
prev_vehicle=$4
prev_lat=$8
prev_lon=$7
}' /stash/dataeng/trimet_data/s23_archive/top75/the75v.csv | head -20
91 4036 20617 45.492492 -122.647742
173 4036 21047
174 4036 33874 45.533173 -122.69475
496 4036 35146
497 4036 35151 45.532878000000004 -122.656595
602 4036 45015
603 4036 45020 45.535340000000005 -122.694802
885 4036 45145 45.535965000000004 -122.698898
1241 4036 46946 45.535573 -122.695
1310 4036 47676
1311 4036 47681 45.525082 -122.681063
1597 4036 47811 45.525217 -122.674002
1686 4036 67997 45.534411999999996 -122.49548500000002
1864 4036 37538 45.534507 -122.476123
1953 4036 48561 45.534977000000005 -122.641023
2220 4036 38107 45.538462 -122.40823799999998
2309 4036 49220 45.533673 -122.60513799999998
2398 4036 65286 45.533482 -122.44786299999998
2487 4036 25637 45.53366 -122.46765500000001
2665 4036 49948 45.533482 -122.560998
srilakp@busdata:/stash/dataeng/trimet_data/Summer2025$
```

Task 4:

Code:

```
awk -F',' 'NR>2 && $4==prev_vehicle && $3 < prev_date {  
    print NR, $4, $3, prev_date  
}  
{  
    prev_vehicle=$4  
    prev_date=$3  
}' /stash/dataeng/trimet_data/s23_archive/top75/the75v.csv | head -20
```

Explanation:

Dates clearly go backward for multiple vehicles

For example, vehicle 3235 — after seeing records from 20JAN2023, the next records jump back to 09JAN2023. That is 11 days backward

Vehicle 3905 — after 15JAN2023, records jump back to 10JAN2023

This happens across hundreds of thousands of rows (rows 513246 all the way to 530621 for just one vehicle)

Therefore, dates are going backward in old as well as new data.

```
srilakp@busdata:/stash/dataeng/trimet_data/Summer2025$ awk -F',' 'NR>2 && $4==prev_vehicle && $3 < prev_date {  
    print NR, $4, $3, prev_date  
}  
{  
    prev_vehicle=$4  
    prev_date=$3  
}' /stash/dataeng/trimet_data/s23_archive/top75/the75v.csv | head -20  
513246 3235 09JAN2023:00:00:00 20JAN2023:00:00:00  
514579 3235 09JAN2023:00:00:00 20JAN2023:00:00:00  
515913 3235 09JAN2023:00:00:00 20JAN2023:00:00:00  
517295 3235 09JAN2023:00:00:00 20JAN2023:00:00:00  
518593 3235 09JAN2023:00:00:00 20JAN2023:00:00:00  
519904 3235 09JAN2023:00:00:00 20JAN2023:00:00:00  
521333 3235 09JAN2023:00:00:00 20JAN2023:00:00:00  
522674 3235 09JAN2023:00:00:00 20JAN2023:00:00:00  
523742 3235 09JAN2023:00:00:00 20JAN2023:00:00:00  
524542 3235 09JAN2023:00:00:00 20JAN2023:00:00:00  
525788 3235 09JAN2023:00:00:00 20JAN2023:00:00:00  
527032 3235 09JAN2023:00:00:00 20JAN2023:00:00:00  
528108 3235 09JAN2023:00:00:00 20JAN2023:00:00:00  
529354 3235 09JAN2023:00:00:00 20JAN2023:00:00:00  
530621 3235 09JAN2023:00:00:00 20JAN2023:00:00:00  
2724800 3905 10JAN2023:00:00:00 15JAN2023:00:00:00  
2725779 3905 10JAN2023:00:00:00 15JAN2023:00:00:00  
2726614 3905 10JAN2023:00:00:00 15JAN2023:00:00:00  
2727415 3905 10JAN2023:00:00:00 15JAN2023:00:00:00  
2733353 3905 10JAN2023:00:00:00 15JAN2023:00:00:00  
srilakp@busdata:/stash/dataeng/trimet_data/Summer2025$
```

Task 5:

Code:

```
head -n 1 /stash/dataeng/trimet_data/s23_archive/se_updated/se_2022-12-05.csv
```

```
awk -F',' 'NR>1 && $12 > 600 {  
    print NR, $2, $12  
}' /stash/dataeng/trimet_data/s23_archive/se_updated/se_2022-12-05.csv | head -20
```

```
awk -F',' 'NR>1 && $12 > 900 {count++} END {print "Dwell > 15 min:", count+0}'  
/stash/dataeng/trimet_data/s23_archive/se_updated/se_2022-12-05.csv
```

Explanation:

**What I checked:** I looked at the DWELL column (column 12) in the old Stop Events data for Dec 5, 2022. DWELL is the time in seconds a bus stays at a stop. Normally this should be a short amount of time — just enough for passengers to board and exit. I flagged anything over 600 seconds (10 min) as suspicious, and anything over 900 seconds (15 min) as a clear anomaly.

**What I found in Old SE (22-23):**

- 25 records in just one day's file have dwell time over 15 minutes
- Some records are extremely high — for example 3092 seconds (over 51 minutes) and 2491 seconds (over 41 minutes) in a single day
- This means buses were recorded as sitting at a stop for very long periods which is not normal operation

**What I found in New CAD/AVL (Summer2025):**

- Same issue found in week 22 — buses appearing to stay at stops for 15-20 minutes regularly
- Flagged as a possible data recording error

**Conclusion:**

- High dwell time anomaly exists in **both old and new data** — not a new problem
- The old data actually shows even more extreme values (51 min, 41 min) compared to the 15-20 min seen in new data
- This could be a real operational event (bus waiting at a terminal) or a recording error where the stop event was not closed properly



```

srilakp@busdata:/stash/dataeng/trimet_data/Summer2025$
srilakp@busdata:/stash/dataeng/trimet_data/Summer2025$
srilakp@busdata:/stash/dataeng/trimet_data/Summer2025$ head -n 1 /stash/dataeng/trimet_data/s23_archive/se_updated/se_2022-12-05
.csv
awk -F',' 'NR>1 && $12 > 600 {
    print NR, $2, $12
}' /stash/dataeng/trimet_data/s23_archive/se_updated/se_2022-12-05.csv | head -20
PDX_TRIP,SERVICE_DATE,VEHICLE_NUMBER,LEAVE_TIME,TRAIN,ROUTE_NUMBER,DIRECTION,SERVICE_KEY,TRIP_NUMBER,STOP_TIME,ARRIVE_TIME,DWELL
, LOCATION_ID, DOOR, LIFT, ONS, OFFS, ESTIMATED_LOAD, MAXIMUM_SPEED, TRAIN_MILEAGE, PATTERN_DISTANCE, LOCATION_DISTANCE, X_COORDINATE, Y_COO
RDINATE, DATA_SOURCE, SCHEDULE_STATUS
5067 05DEC2022:00:00:00 897
14117 05DEC2022:00:00:00 686
19151 05DEC2022:00:00:00 732
30541 05DEC2022:00:00:00 1380
30542 05DEC2022:00:00:00 1865
40348 05DEC2022:00:00:00 1364
40547 05DEC2022:00:00:00 1093
46779 05DEC2022:00:00:00 1402
49657 05DEC2022:00:00:00 837
56342 05DEC2022:00:00:00 1815
66970 05DEC2022:00:00:00 844
72721 05DEC2022:00:00:00 1523
80734 05DEC2022:00:00:00 1445
81427 05DEC2022:00:00:00 1093
81533 05DEC2022:00:00:00 1146
107930 05DEC2022:00:00:00 1008
145211 05DEC2022:00:00:00 736
145412 05DEC2022:00:00:00 2491
157618 05DEC2022:00:00:00 1010
157980 05DEC2022:00:00:00 3092
srilakp@busdata:/stash/dataeng/trimet_data/Summer2025$
srilakp@busdata:/stash/dataeng/trimet_data/Summer2025$
srilakp@busdata:/stash/dataeng/trimet_data/Summer2025$ awk -F',' 'NR>1 && $12 > 900 {count++} END {print "Dwell > 15 min:", coun
t+0}' /stash/dataeng/trimet_data/s23_archive/se_updated/se_2022-12-05.csv
Dwell > 15 min: 25
srilakp@busdata:/stash/dataeng/trimet_data/Summer2025$
srilakp@busdata:/stash/dataeng/trimet_data/Summer2025$

```

## Task 6:

### Code:

```

awk -F',' 'NR>1 && $16==0 && $17==0 {count++} END {print "Stops with no passenger activity:", count+0}' \
/stash/dataeng/trimet_data/s23_archive/se_updated/se_2022-12-05.csv

```

```

awk -F',' 'NR>1 {total++} END {print "Total records:", total+0}' \
/stash/dataeng/trimet_data/s23_archive/se_updated/se_2022-12-05.csv

```

### Explanation:

**What I checked:** I looked at ONS (column 16 — passengers boarding) and OFFS (column 17 — passengers alighting) in the old Stop Events data for Dec 5, 2022. If both are 0, it means no passenger activity was recorded at that stop.

### What I found in Old SE (22-23):

- 181,984 out of 273,439 records have no passenger activity
- That is **66.6%** of all stops showing zero boardings and zero alightings in a single day

### What I found in New CAD/AVL (Summer2025):

- 320,103 out of 500,000 records had no passenger activity
- That is **64%** of stops

### Conclusion:

- No passenger activity issue exists in **both old and new data** — not a new problem
- The percentage is almost identical — 66.6% in old data vs 64% in new data
- This is likely normal for transit data — many stops on a route will have no passengers at certain times of day, especially late night or low-demand stops
- However the high percentage still means either passenger counts are not being recorded properly, or many stop events are being logged even when the bus passes through without stopping

```

srilakp@busdata:/stash/dataeng/trimet_data/Summer2025$ awk -F',' 'NR>1 && $16==0 && $17==0 {count++} END {print "Stops with no passenger activity:", count+0}' \
/stash/dataeng/trimet_data/s23_archive/se_updated/se_2022-12-05.csv

awk -F',' 'NR>1 {total++} END {print "Total records:", total+0}' \
/stash/dataeng/trimet_data/s23_archive/se_updated/se_2022-12-05.csv
Stops with no passenger activity: 181984
Total records: 273439
srilakp@busdata:/stash/dataeng/trimet_data/Summer2025$

```

Task 7:

**Task 1** – The missing field problems I found in Summer2025, do they also exist in the old 22-23 data?

I first found 4 missing ARRIVE\_DATETIME values in Summer2025. When I checked the old 22-23 data, this field didn't even exist there. Old data used ACT\_TIME which was always fully filled in, so this is a new field introduced in Summer2025. Interestingly, the old 22-23 data had 7,059 missing GPS coordinates (lat/lon), but Summer2025 has zero missing GPS values, meaning GPS recording actually improved in the new data. So overall, the missing field issue in Summer2025 is very minor (only 4 rows out of 1.1 million) and the new data is actually cleaner than the old data in terms of GPS completeness.

**Task 2** – The time ordering problem I found in Summer2025 (records for the same bus not appearing in chronological order), does it also exist in the old 22-23 data?

For the same bus, records don't always appear in the order they happened. An earlier time sometimes shows up after a later time in the file. This exists in both old and new data. It is not something new that was introduced.

**Task 3** – The unrealistic GPS location jumps I found in Summer2025 (a bus appearing to move several kilometers in one second), do they also exist in the old 22-23 data?

A bus cannot travel 4-5 km in one second, but the data shows exactly that. The GPS coordinate jumps by several kilometers between back-to-back records. This happens in both old and new data. Not a new problem.

**Task 4** – The dates going backward for the same bus that I found in Summer2025, does it also happen in the old 22-23 data?

For the same bus, the dates in the file sometimes go backward. For example, a record from January 20 is followed by a record from January 9. This happens in both old and new data across hundreds of thousands of rows.

**Task 5** – The high dwell time anomaly I found in Summer2025 (buses recorded as sitting at a stop for 15-20 minutes or more), does it also exist in the old 22-23 data?

The data shows some buses sitting at a stop for 15-20 minutes or more, which is unusual. The old data is actually worse. Some records show 41 and 51 minute stops in a single day. Same issue exists in the new data. Could be a real terminal wait or a recording error.

**Task 6** – The high percentage of stops with no passenger activity I found in Summer2025, does it also exist in the old 22-23 data?

About 64-66% of stop records show zero passengers boarding or alighting in both old and new data. The numbers are almost identical. 66.6% in old data, 64% in new data. This is likely normal for low-demand stops and late-night runs, but worth noting.